UNIVERSIDAD RAFAEL LANDÍVAR

Facultad de Ingeniería - Ingeniería en Informática y Sistemas
Campus San Alberto Hurtado, S.J. de Quetzaltenango
Noviembre de 2026

DOCUMENTO TÉCNICO

Laboratorio No. 2 - Manejo e Implementación de Archivos: Agenda de Contactos con Acceso Aleatorio, Índice Primario, Verificación SHA-256 y Respaldo ZIP

Jonathan André Soto Tzul
1533825

1. Introducción

El presente documento describe el diseño técnico, la arquitectura y la implementación de una aplicación de escritorio desarrollada en Python con la biblioteca Tkinter para la administración de una agenda de contactos. El objetivo principal es implementar un sistema de almacenamiento persistente basado en registros binarios de tamaño fijo (150 bytes), permitiendo operaciones de acceso aleatorio directo mediante las primitivas seek() y tell() del sistema de archivos. Para optimizar los tiempos de consulta, se integra un índice primario almacenado en formato JSON que mapea identificadores únicos hacia posiciones relativas de registro en disco. Adicionalmente, se implementa verificación criptográfica de integridad de datos mediante hash SHA-256, un motor de respaldo en archivos ZIP con cálculo de compresión, exportación de registros a formato JSON y generación de reportes profesionales en PDF.

2. Comparación y justificación del formato de almacenamiento

Para la persistencia de los contactos del sistema se evaluaron dos alternativas principales de almacenamiento en disco:

Criterio	Formato Seleccionado (Binario de Tamaño Fijo + Índ	Formato Descartado (Texto Plano / CSV / JSON Puro)
Estructura de Datos	Registros empaquetados exactamente a 150 bytes con struct (>i, 50s, 16s, 50s, 30s).	Longitud variable delimitada por comas, saltos de línea o sintaxis JSON.
		Acceso Aleator

Justificación Técnica: Se eligió el formato binario de registros de tamaño fijo debido a que permite aprovechar al máximo las capacidades de acceso aleatorio de los sistemas operativos modernos. Al fijar cada registro en 150 bytes exactos, el desplazamiento en bytes de cualquier registro N se calcula de forma inmediata como offset = N * 150. Esto hace posible actualizar un contacto en milisegundos mediante seek() y reescritura puntual, mientras que en un archivo CSV o JSON secuencial sería indispensable reescribir la totalidad de los datos en disco, ocasionando un costo de I/O cuadrático al escalar el volumen de información.

3. Implementación del acceso aleatorio, empaquetado e índice

La estructura del registro se compone de 5 campos claramente dimensionados para totalizar 150 bytes binarios:
- ID: 4 bytes (entero con signo en orden big-endian, formato >i)

- Nombre: 50 bytes (cadena UTF-8 ajustada y rellenada con nulos)
- Teléfono: 16 bytes (cadena UTF-8)
- Correo electrónico: 50 bytes (cadena UTF-8)
- Ciudad: 30 bytes (cadena UTF-8)
Total por registro: 4 + 50 + 16 + 50 + 30 = 150 bytes exactos.

Código de empaquetado y desempaquetado (core/record.py):

```
TAMANO_REGISTRO = 150
FORMATO = f">i{NOMBRE_LEN}s{TELEFONO_LEN}s{CORREO_LEN}s{CIUDAD_LEN}s"

def empaquetar_registro(contacto: dict) -> bytes:
    id_val = int(contacto["id"])
    nom_b = _ajustar_campo(contacto.get("nombre", ""), NOMBRE_LEN)
    tel_b = _ajustar_campo(contacto.get("telefono", ""), TELEFONO_LEN)
    cor_b = _ajustar_campo(contacto.get("correo", ""), CORREO_LEN)
    ciu_b = _ajustar_campo(contacto.get("ciudad", ""), CIUDAD_LEN)
    return struct.pack(FORMATO, id_val, nom_b, tel_b, cor_b, ciu_b)

def desempaquetar_registro(datos: bytes) -> dict:
    id_val, nom_b, tel_b, cor_b, ciu_b = struct.unpack(FORMATO, datos)
    return {
        "id": id_val,
        "nombre": _decodificar_campo(nom_b),
        "telefono": _decodificar_campo(tel_b),
        "correo": _decodificar_campo(cor_b),
        "ciudad": _decodificar_campo(ciu_b),
    }
```

Acceso aleatorio con seek() y tell() (core/manager_contact.py):

```
def buscar_por_id(id_contacto: int) -> tuple[dict, dict]:
    indice = cargar_indice()
    if id_contacto not in indice:
        raise KeyError(f"No existe contacto con ID {id_contacto}")

    num_registro = indice[id_contacto]
    offset = num_registro * TAMANO_REGISTRO

    with open(RUTA_DAT, "rb") as f:
        f.seek(offset)
        posicion = f.tell()
        chunk = f.read(TAMANO_REGISTRO)

    contacto = desempaquetar_registro(chunk)
    return contacto, {"registro": num_registro, "posicion": posicion}
```

4. Manejo de errores y tolerancia a fallos

El sistema fue diseñado bajo el principio de robustez y degradación controlada: la capa core/ lanza excepciones estándar de Python cuando se producen anomalías de archivo o de datos, mientras que la capa gui/ las intercepta y las traduce en explicaciones comprensibles mediante el módulo mensajes.py, evitando cierres inesperados de la aplicación:

```
def formatear_error(error: Exception) -> str:
    if isinstance(error, FileNotFoundError):
        return f"No se encontró el archivo:\n{getattr(error, 'filename', str(error))}"
    elif isinstance(error, PermissionError):
        return f"Permiso denegado para acceder al archivo:\n{getattr(error, 'filename', str(error))}"
    elif isinstance(error, json.JSONDecodeError):
        return f"El archivo JSON de índice está corrupto:\n{error.msg} (línea {error.lineno})."
    elif isinstance(error, KeyError):
        return f"El registro solicitado no existe: {error}"
    elif isinstance(error, ValueError):
        return str(error)
    elif isinstance(error, OSError):
        return f"Error del sistema operativo al acceder al archivo:\n{error}"
    return f"Ocurrió un error inesperado:\n{error}"

def mostrar_error(error: Exception, titulo: str = "Error", parent=None):
    messagebox.showerror(titulo, formatear_error(error), parent=parent)
```

5. Estructura de Clases y Arquitectura del Sistema

La arquitectura de la aplicación mantiene un desacoplamiento estricto entre el motor de archivos y la interfaz:

Módulo / Archivo	Responsabilidad y Contenido Técnico
main.py	Punto de entrada de la aplicación. Inicializa MainWindow y arranca el loop de eventos Tkinter.
core/record.py	Define el formato binario (150 bytes), empaquetado y desempaquetado de registros con struct.
core/manager_contact.py	Operaciones CRUD en contact.dat con seek() y tell(). Coordina índice primario e integridad.
core/index.py	Gestión del índice primario index.json (mapeo en memoria y serialización ID -> num_registro).
core/integrity.py	Cálculo, almacenamiento y verificación de hash SHA-256 sobre contact.dat (contacts.sha256).
core/backup.py	Generación de respaldo ZIP con contact.dat, index.json y sha256; cálculo de compresión.
core/exporter.py	Exportación de datos a contact.json y generación de reportes impresos en reporte.pdf con fpdf2.
core/file_info.py	Consulta de metadatos del archivo con os.stat (tamaño, mtime) y permisos con os.access.
gui/main_window.py	Ventana principal. Despliega barra de menús, botones de acceso directo y barra de estado SHA-256.
gui/window_contact.py	Ventana modal con pestañas: Agregar Contacto, Buscar por ID y Modificar con carga previa.
gui/window_info.py	Ventana de información del archivo; muestra tamaño en bytes/KB, fecha formateada y permisos.
gui/mensajes.py	Capa de traducción de errores y despliegue de alertas modales messagebox para el usuario.

6. Evidencia del Proceso de Pruebas y Funcionamiento del Sistema

Etapa 1 - Ventana Principal y Menú de Opciones

La aplicación arranca cargando la ventana principal con su barra superior y accesos directos agrupados por categorías: Contactos, Exportar y Herramientas. De forma asíncrona a los 300 ms de iniciar, se ejecuta automáticamente la verificación de integridad criptográfica SHA-256 del archivo contact.dat contra contacts.sha256, actualizando la etiqueta inferior con el estado ('correcta').

Etapa 2 - Registro de Nuevos Contactos (Pestaña Agregar)

El usuario introduce el ID entero único y los datos del contacto (Nombre, Teléfono, Correo y Ciudad). La función

core.manager_contact.agregar_contacto() valida que el ID no exista en index.json, calcula el número de registro correlativo mediante el tamaño de archivo dividido entre 150 bytes, posiciona el puntero al final con seek() y escribe los 150 bytes empaquetados. La interfaz confirma la operación reportando el número de registro y el desplazamiento exacto devuelto por tell().

Etapa 3 - Búsqueda Directa por ID (Pestaña Buscar)

Al ingresar un ID numérico, el sistema consulta en O(1) el número de registro en index.json. Inmediatamente salta al byte exacto mediante f.seek(num_registro * 150), lee el bloque de 150 bytes y lo desempaqueta, mostrando en pantalla los datos recuperados junto al número de registro y su posición en bytes.

Etapa 4 - Modificación In-Situ de Registro (Pestaña Modificar)

Se busca primero el contacto para cargar sus valores actuales en las entradas editables. Al presionar 'Guardar cambios', la función modificar_contacto() realiza un seek() a la posición calculada y sobrescribe exactamente los 150 bytes del registro en contact.dat sin tocar ningún otro contacto contiguo. Posteriormente recalcula y actualiza automáticamente el hash en contacts.sha256.

Etapa 5 - Verificación de Integridad SHA-256 (Detección de Modificación Externa)

Al ejecutar 'Verificar integridad' desde el menú Herramientas, el sistema recalcula el hash SHA-256 de contact.dat leyendo por bloques de 64 KB y lo compara con el texto almacenado en contacts.sha256. Si el archivo fue editado por fuera de la aplicación (por ejemplo con un editor hexadecimal), el sistema detecta la discrepancia inmediatamente y lanza un mensaje modal de advertencia preventiva.

Etapa 6 - Creación de Respaldo Comprimido en ZIP (Backup)

La opción 'Crear respaldo ZIP' empaqueta los archivos contact.dat, index.json y contacts.sha256 en el archivo comprimido backups/backup.zip utilizando zipfile.ZIP_DEFLATED. La aplicación cuantifica el tamaño original acumulado contra el tamaño del ZIP resultante y presenta al usuario la tasa porcentual de compresión obtenida.

Etapa 7 - Exportación a Formato JSON y Generación de Reporte PDF

El módulo core.exporter exporta la totalidad de los contactos ordenados por ID hacia exports/contact.json. A partir de este JSON, la biblioteca fpdf2 construye exports/reporte.pdf aplicando encabezados institucionales, numeración de páginas dinámica ('Página X/5') y una tabla estilizada con sombreado alterno de filas.

Etapa 8 - Información del Archivo mediante Metadatos del Sistema Operativo

La ventana 'Información del archivo' realiza llamadas a os.stat() para recuperar la ruta absoluta, el tamaño exacto en bytes y KB, y el timestamp de última modificación. Asimismo, utiliza os.access() para comprobar y mostrar si la aplicación posee permisos de lectura (os.R_OK) y escritura (os.W_OK).

7. Conclusiones

1. Acceso Aleatorio O(1) y Eficiencia de Disco: El empleo de registros con tamaño constante y predeterminado (150 bytes) garantiza la posibilidad de realizar saltos exactos a cualquier posición del archivo mediante seek(). Esto elimina la necesidad de escanear el archivo de manera secuencial o de reescribirlo por completo al modificar datos, optimizando el rendimiento en operaciones de entrada/salida.

2. Desacoplamiento de Arquitectura e Indexación: La combinación de un índice primario en memoria (index.json) con un archivo binario en disco ofrece un equilibrio óptimo entre velocidad de búsqueda algorítmica y persistencia durable. Asimismo, el diseño modular separa estrictamente la lógica de almacenamiento (core/) de la presentación visual (gui/), asegurando alta mantenibilidad.

3. Seguridad, Integridad y Disponibilidad: La integración del algoritmo criptográfico SHA-256 permite autenticar en cualquier momento la legitimidad de los datos frente a alteraciones externas o corrupciones de hardware. Sumado al mecanismo de respaldo comprimido en ZIP, se garantiza la resiliencia y capacidad de recuperación del sistema ante pérdidas imprevistas.